

Agentic SOV Intelligence System

Four collaborating AI agents that turn chaotic Statements of Values into schema-perfect, audit-ready exposure data, with a human approving every change.



TEAM

Cache Me If You Can

BIT Mesra

TEAM LEAD

Dishan Sarkar

BTECH/10614/23 · CSE (AIML)

MEMBER

Akhand Awasthi

BTECH/10595/23 · ECE

01 • THE PROBLEM

Every client SOV is a new schema, cleaned by hand

A Statement of Values catalogues every insured asset: location, construction, occupancy and insured values. It feeds the exposure models that set maximum financial risk, pricing and underwriting strategy.

Clients send SOVs with different sheet layouts, headers on arbitrary rows, merged cells, abbreviations and malformed values. Exposure Management teams clean each one manually.

Bad SOV data → wrong exposure → mispriced policies → underwriting losses.

53%

Header mapping accuracy of today's manual baseline

Hours

Of analyst time spent cleansing a single SOV

17

Strictly typed fields the exposure models require

Zero

Standardised cleansing procedures in place today

Five obstacles that simple parsing cannot solve



Structural chaos

Varying sheet counts, header rows anywhere, merged cells and no shared naming convention.



Semantic gap

“Bldg Repl Cost New” means Building Value. “Fire Prot.” means Fire Sprinklers (Y/N).



LLM reliability

Models can hallucinate mappings or invent values. Every AI suggestion must be verified.



Schema strictness

A wrong name, type or missing field corrupts the downstream risk calculation.



Scale

Thousands of rows across dozens of sheets, processed quickly and without memory failures.

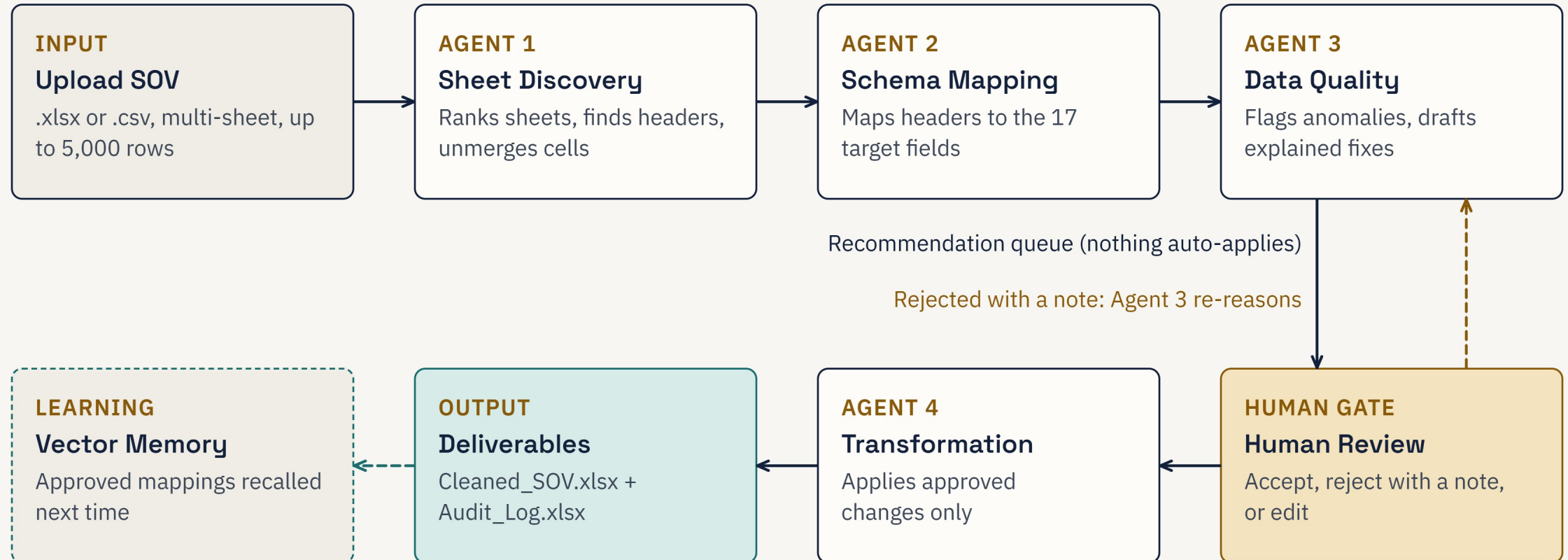
The answer is an engineered system: AI proposes, deterministic code verifies, and a human approves before anything changes.

OUR DESIGN PRINCIPLE

**AI proposes.
Code verifies.
Humans approve.
Everything is audited.**

The LLM never writes a single cell. It reasons, explains, and selects from a whitelist of deterministic, tested transformations that run only after a person says yes.

One upload, four agents, one audit-ready file



Five layers, each replaceable on its own

Presentation	Streamlit review console	Live agent workflow graph	Before / after data preview	Download: SOV + audit log
Orchestration	LangGraph StateGraph	Typed SOVState (Pydantic)	interrupt() human gate	SQLite checkpointer
Agents	1 · Sheet Discovery	2 · Schema Mapping	3 · Data Quality & Reasoning	4 · Controlled Transform
AI services	RapidFuzz fuzzy matcher	bge-small embeddings (CPU)	LLM gateway: Groq / Ollama	Rule + validation engine
Data layer	pandas + openpyxl I/O	ChromaDB vector memory	Audit store (JSON / XLSX)	Ephemeral session files

Cross-cutting: .env secrets · structured logging · pytest ground-truth harness · provider-agnostic LLM interface

Find the real data before touching it

- 1 Profile every sheet**
Row volume, column richness, null ratio, numeric density and merged-cell ranges, read with openpyxl.
- 2 Locate the header row**
Score the first 30 rows on text ratio, fill ratio, hits against a 17-field synonym vocabulary, and typed data below.
- 3 Repair the structure**
Unmerge and forward-fill merged headers, collapse multi-row headers, drop blank and total rows.
- 4 Classify and explain**
Primary, Secondary or Reject, with a weighted score and plain-English reasons.

Example output (illustrative)

```
{  
  "sheet_name": "Location Schedule",  
  "classification": "Primary",  
  "header_row": 3,  
  "confidence": 0.94,  
  "reasoning": [  
    "11 of 14 headers match vocabulary",  
    "Typed numeric data below row 3",  
    "4% nulls, 1,250 contiguous rows"  
  ]  
}
```

Initial weights: header match 0.35 · data density 0.25 · type consistency 0.20 · row volume 0.20, tuned on the sample SOVs.

A five-stage cascade: cheap and certain first, LLM last

Each header stops at the first stage that resolves it confidently, so the LLM only sees the genuinely ambiguous ones.

Stage	Technique	Example	Confidence
0 · Memory	Recall human-approved mappings from ChromaDB	“Bldg Repl Cost New” seen before	Human-verified
1 · Exact	Normalised match: case, punctuation, spacing	“Street Address” → Address	1.00
2 · Fuzzy	Abbreviation expansion + RapidFuzz ≥ 0.75	“Yr Blt” → Year Built	Fuzzy score
3 · Semantic	bge-small embeddings: header vs field definitions	“Fire Prot.” → Fire Sprinklers (Y/N)	Calibrated cosine
4 · LLM	Top-3 candidates + sample values → enum-constrained JSON	“Loc #” → Reference	Blended with evidence

Confidence weighs the header and the data itself

Confidence score

`conf = 0.45 × name_similarity`
`+ 0.35 × value_profile_fit`
`+ 0.20 × method_agreement`

Memory hits from human-approved mappings get a boost, capped at 0.99. Initial weights, tuned on the three sample SOVs.

≥ 0.90

Eligible for “Approve All”

0.50 – 0.89

Reviewed item by item

< 0.50

human_review_required flag

Pattern in the values	Implies
Integers 1800–2026	Year Built
Only Y / N / Y13 / Y(13R) codes	Fire Sprinklers (Y/N)
5-digit codes, leading zeros	Zip
2-letter codes in the state set	State
Large currency-like decimals	Building Value, Contents, BI, Other
Small integers, 1–100	Storeys

A Hungarian assignment keeps mappings one-to-one: two columns can never both become Building Value.

Rules find the issues. The LLM explains them.

Check	Rule	Example flag	Severity
Completeness	Non-null % for every mapped field	County 62% complete	Medium
Data type	Numeric / integer per data dictionary	Storeys = “two”	High
Logical	TIV ≥ 0, Year Built ≤ now, Storeys ≥ 1	Year Built = 2031	High
Format	Currency symbols, mixed dates, state names	“\$1,250,000”, “Calif.”	Low
Allowed values	Sprinklers must be Y, N, Y13 or Y(13R)	“Yes”, “13R”	Medium
Duplicates	Repeated Reference IDs across rows	Ref 104 appears twice	Medium



LLM reasoning runs on grouped patterns, not on every row: one call explains “currency symbols in Building Value (212 rows)”, picks a whitelisted fix and states what it is unsure about.

Every suggestion is explained, and none apply on their own

STANDARDISATION

Confidence 0.96

Illustrative

Building Value has currency symbols in 212 rows

Before

\$1,250,000



After

1250000.0

Why: the data dictionary types Building Value as Float, and “\$” and “,” block casting.

Operation: strip_currency → to_float.

Uncertain: 3 rows read “TBD”. These stay blank and are flagged, never guessed.

Accept

Reject with note

Edit

Four action types

column_mapping

Rename a raw header to a target field

data_correction

Fix a malformed value, e.g. “two” → 2

standardisation

Normalise currency, states, codes

flag_for_review

Unsafe to automate: a person decides

Six guardrails against hallucination and silent edits

01

The LLM never edits data

It only selects an operation from a tested whitelist (strip_currency, to_int, state_to_abbrev). Code applies it.

02

Constrained output

Pydantic-validated JSON: a target must be one of the 17 fields or null. Invalid output is retried, then escalated.

03

Evidence cross-check

A mapping that conflicts with the column's value profile drops below 0.50 and goes to a person.

04

Nothing is invented

Imputation is not an allowed operation. Missing values stay blank in the output, as C-02 requires.

05

Hard human gate

LangGraph interrupt() pauses before Agent 4. Export stays locked until every flagged item is decided.

06

Complete audit trail

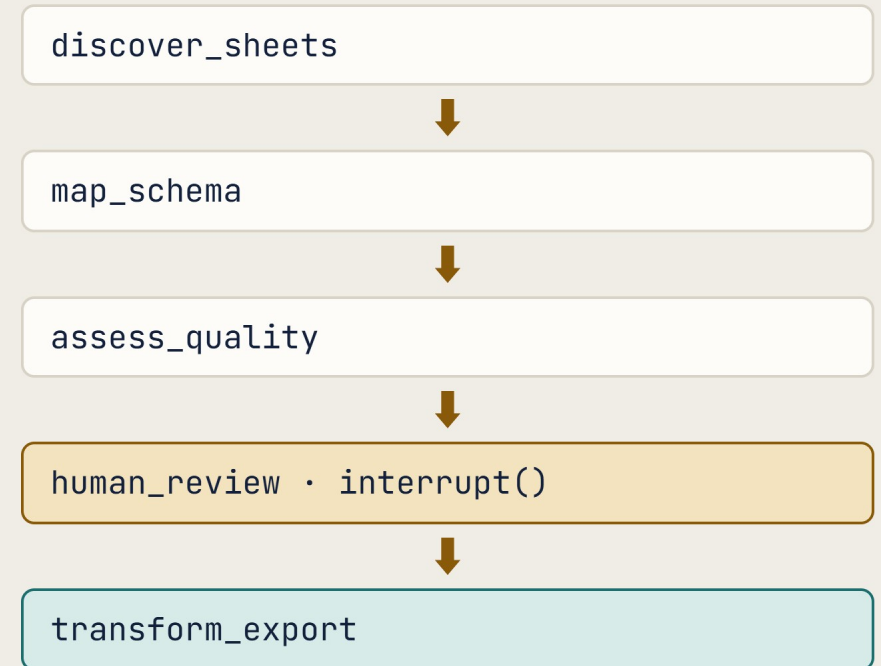
Source, target, operation, before, after, confidence, approver and timestamp for every applied change.

One typed state object, validated at every hand-off

Shared state (Pydantic model)

```
class SOVState(BaseModel):  
    file_meta: FileMeta  
    sheet_manifest: list[SheetScore]      # Agent 1  
    header_row: int                       # Agent 1  
    mappings: dict[str, Mapping]          # Agent 2  
    quality_report: QualityReport         # Agent 3  
    recommendations: list[Recommendation] # Agent 3  
    decisions: list[ReviewDecision]       # Reviewer  
    audit_log: list[AuditEntry]           # Agent 4  
    stage: Stage                         # Orchestrator
```

Each agent writes only its own keys; everything else is read-only to it. State is checkpointed, so a review session can pause and resume.



Gates validate each node's keys before the next runs (NFR-7). A rejection routes back to `assess_quality`.

A review console a non-technical underwriter can use

≥ 0.90 · Approve All

High-confidence items cleared in one click, and still logged.

0.50 – 0.89 · Review each

Rationale, evidence and before/after shown side by side.

< 0.50 · Must review

Export stays locked until every flagged item is decided.

Also in the console: mapping dropdown for edits, live agent workflow graph, and an SOV quality score at intake.

1. Reviewer rejects with a note, e.g. “Fire Prot. is a protection class here”



2. Agent 3 re-reasons with the note, the original evidence and value profile



3. A revised recommendation returns to the queue with a new rationale



4. Rejected again? Escalated as flag_for_review for a manual decision

Schema-perfect output, with a receipt for every change

The Cleaned_SOV.xlsx contract

- ✓ File Cleaned_SOV.xlsx, sheet “Cleaned_SOV”
- ✓ Exactly 17 columns, dictionary order, exact names
- ✓ Headers in row 1, data from row 2
- ✓ No merged cells, no colour formatting
- ✓ Types cast to Float, Integer or String per field
- ✓ Missing values stay blank, never 0 or “N/A”
- ✓ Pandera schema check must pass before download
- ✓ Audit_Log.xlsx and a processing summary report

Audit_Log entry (illustrative)

```
{  
  "source_column": "Bldg Repl Cost",  
  "target_column": "Building Value",  
  "transformation": "strip_currency → to_float",  
  "before_value": "$1,250,000",  
  "after_value": 1250000.0,  
  "confidence": 0.96,  
  "approved_by": "reviewer.01",  
  "timestamp": "2026-10-03T14:22:08Z"  
}
```

Proven, free-tier tools that run on a laptop

Layer	Choice	Why
Review UI	Streamlit + live LangGraph workflow view	Usable approval console within 24 hours
Orchestration	LangGraph · interrupt() · SQLite checkpoints	Native human-in-the-loop pause/resume
LLM	Groq Llama 3.3 70B · Ollama local fallback	Fast structured output; on-prem option
Embeddings	sentence-transformers, bge-small-en-v1.5	CPU-friendly semantic header matching
Fuzzy match	RapidFuzz (token-set ratio)	Fast, proven string similarity
Excel I/O	pandas + openpyxl	Merged cells, multi-sheet, exact output
Validation	Pydantic v2 + Pandera	Typed agent contracts, DataFrame schema
Vector memory	ChromaDB (embedded, persistent)	Learns from approved mappings locally
Quality & ops	pytest harness · python-dotenv · Git	Measured accuracy; no hard-coded keys

All LLM calls go through one provider-agnostic interface: GPT-4o is a one-line configuration swap.

Built for sensitive commercial insurance data



Data minimisation

Only headers, field definitions and up to 5 masked sample values reach the LLM, never full rows or addresses.



Local-first option

Ollama mode keeps inference on-premise. Embeddings and fuzzy matching always run locally.



Secrets hygiene

API keys come only from environment variables (.env, git-ignored). None are hard-coded.



Ephemeral processing

Uploads live in a per-session temp store and are purged after export.



AI has no data access

The LLM has no tools and no file access. It sees prompts; code owns the data.



Traceable decisions

Every AI suggestion and human decision is logged with reviewer and timestamp.

Engineered against the judging rubric

Criterion	Weight	Target	How we hit it
Mapping accuracy	20%	≥ 74% (baseline 53%)	5-stage cascade + value profiling + memory
Data quality detection	15%	≥ 90% anomaly recall	Deterministic rule engine, per-row flags
Transformation correctness	15%	≥ 95% rows match truth	Whitelisted ops + Pandera validation
Agentic design	20%	4 clearly distinct agents	LangGraph nodes, typed state, gates
Explainability	15%	100% rationale coverage	Rationale is a required schema field
Human-in-the-loop UX	10%	Usable by non-experts	Confidence bands, before/after preview
Innovation & bonus	5%	All 4 bonus items	Memory, live graph, re-reasoning, score

Day one: we hand-label the 3 sample SOVs as ground truth, and an automated harness scores every commit against these targets.

What changes for the insurer

< 60 s

Agentic pipeline plus a brief human review,
instead of hours per SOV (PS target)

Business

- Hours of cleansing become minutes
- Accurate exposure data, correctly priced policies
- More submissions without adding headcount

≥ 74%

Header mapping accuracy target, up from a
53% manual baseline

Technical

- Reusable multi-agent intake pattern
- Guardrails make LLM output safe to act on
- Every approval improves the vector memory

100%

Of applied transformations recorded in the
audit log

Organisational

- Analysts move from data cleanup to judgment
- Consistent results across analysts and teams
- Audit-ready trail builds trust in AI

Flexible like an LLM, safe like a rules engine

Capability	Manual cleansing	Rule / template ETL	Generic LLM chat	Our system
Handles never-seen formats	Slowly	Breaks	Yes	Yes
Explains every decision	Rarely	No	Partly	Always + uncertainty
Never invents data	Yes	Yes	No guarantee	Enforced by design
Human approval before change	Implicit	No	No	Enforced gate
Full audit trail	No	Partial	No	Every change
Gets better with use	Per analyst	Manual rule edits	No	Vector memory
Minutes per SOV	No, hours	Only if template fits	Yes	Yes

24 hours, two parallel tracks

Hours	Dishan Sarkar · Lead (CSE-AIML)	Akhand Awasthi (ECE)	Milestone
0–2	LangGraph skeleton + SOVState	Hand-label 3 SOVs as ground truth	Scoring harness live
2–7	Agent 2 cascade + embeddings	Agent 1 sheet + header detection	Mapping runs on all 3
7–12	Agent 3 LLM reasoning + guardrails	Agent 3 rules + quality report	Targets checked on harness
12–17	Re-reasoning loop + vector memory	Streamlit review console	End-to-end HITL flow
17–21	Agent 4 transform + audit log	Live workflow graph + previews	Validated Cleaned_SOV.xlsx
21–24	Tuning against the harness	Error handling, README, demo	Demo rehearsed

The minimum bar (4 agents, end to end on all 3 samples) comes first; bonus features are layered on only once it works.

What could go wrong, and what we do about it

Risk	Mitigation
LLM rate limits or outage	Cascade resolves easy headers first; Ollama fallback; cached responses
Unusual workbook breaks parsing	Defensive per-sheet parsing and readable errors, never a crash (NFR-4)
Confident but wrong mapping	Value-profile cross-check, < 0.50 escalation, and the human gate
Scope creep in 24 hours	Core pipeline first; bonus features only after end-to-end works
Sensitive data exposure	Masked samples only, local mode, keys in environment variables only

The seven stages judges will see

1	Upload & sheet detection	Multi-sheet sample: Primary sheet and header row identified
2	Schema mapping	JSON with confidences; semantic hit “Bldg Repl Cost” → Building Value
3	Data quality report	Per-field completeness and typed anomaly flags
4	Recommendations	Mapping, correction and standardisation, each with rationale
5	Human approval	Accept one, reject one with a note, watch Agent 3 re-reason
6	Transform + audit	Approved changes applied, audit entries shown alongside
7	Download	Cleaned_SOV.xlsx: 17 headers, correct types, no merged cells

THANK YOU

Messy SOV in. Trusted exposure data out.

Agentic SOV Intelligence System: AI proposes, code verifies, humans approve, and everything is audited.

TEAM

Cache Me If You Can

BIT Mesra

TEAM LEAD

Dishan Sarkar

BTECH/10614/23 · CSE (AIML)

MEMBER

Akhand Awasthi

BTECH/10595/23 · ECE